

Mindful Timer, Part 2: YouTube Tracks + SwiftData History

Continues from the Part 1 timer. Two additions: a track list with thumbnails that plays your YouTube meditation videos, and SwiftData persistence so past sessions are remembered across launches.

Part A — YouTube thumbnails + playback

No third-party packages needed — `WKWebView` can embed YouTube's player directly, and YouTube's thumbnail images are just public URLs (`img.youtube.com/vi/{videoID}/hqdefault.jpg`).

A1. Define your tracks

New file, `MeditationTrack.swift`:

```
import Foundation

struct MeditationTrack: Identifiable {
    let id = UUID()
    let title: String
    let youtubeID: String // the part after "v=" in a YouTube URL
}

let sampleTracks: [MeditationTrack] = [
    MeditationTrack(title: "Morning Calm", youtubeID:
"PUT_YOUR_VIDEO_ID_HERE"),
    MeditationTrack(title: "Deep Focus", youtubeID:
"PUT_YOUR_VIDEO_ID_HERE"),
    MeditationTrack(title: "Evening Wind Down", youtubeID:
"PUT_YOUR_VIDEO_ID_HERE"),
]
```

Swap in your actual video IDs — e.g. for `youtube.com/watch?v=dQw4w9WgXcQ`, the ID is `dQw4w9WgXcQ`.

A2. The player itself

`WKWebView` isn't a native SwiftUI view, so you wrap it — this is the standard pattern for dropping any UIKit view into SwiftUI (`UIViewRepresentable`, roughly analogous to wrapping a vanilla-JS widget in a React component with `useRef`).

New file, `YouTubePlayerView.swift`:

```
import SwiftUI
import WebKit

struct YouTubePlayerView: UIViewRepresentable {
```

```

let videoID: String

func makeUIView(context: Context) -> WKWebView {
    WKWebView()
}

func updateUIView(_ webView: WKWebView, context: Context) {
    guard let url = URL(
        string: "https://www.youtube.com/embed/\(videoID)?
playsinline=1&autoplay=1"
    ) else { return }
    webView.load(URLRequest(url: url))
}
}

```

A3. Thumbnail list

New file, `TrackListView.swift`:

```

import SwiftUI

struct TrackListView: View {
    @State private var selectedTrack: MeditationTrack?

    var body: some View {
        List(sampleTracks) { track in
            Button {
                selectedTrack = track
            } label: {
                HStack(spacing: 12) {
                    AsyncImage(
                        url: URL(string: "https://img.youtube.com/vi/\
(track.youtubeID)/hqdefault.jpg")
                    ) { image in
                        image.resizable().aspectRatio(16/9, contentMode:
.fill)
                    } placeholder: {
                        Color.gray.opacity(0.2)
                    }
                    .frame(width: 120, height: 68)
                    .clipped()
                    .cornerRadius(8)

                    Text(track.title)
                        .foregroundColor(.primary)
                }
            }
        }
        .navigationTitle("Tracks")
        .sheet(item: $selectedTrack) { track in
            YouTubePlayerView(videoID: track.youtubeID)
        }
    }
}

```

```

    }
  }
}

```

`AsyncImage` is SwiftUI's built-in remote-image loader — no `Kingfisher`/`SDWebImage` needed for something this simple, similar to how you might just use `<img src>` in React rather than pulling in a library. `.sheet(item:)` presents the player modally whenever `selectedTrack` becomes non-`nil`, and dismisses when it's set back to `nil`.

A4. Wire it into the app

If you're using the default single-view template, give yourself a tab bar so the timer and tracks live side by side. Replace your app's root view with:

```

struct RootView: View {
    var body: some View {
        TabView {
            ContentView()
                .tabItem { Label("Timer", systemImage: "timer") }

            NavigationStack {
                TrackListView()
            }
                .tabItem { Label("Tracks", systemImage: "music.note.list") }
        }
    }
}

```

...and point your `@main App` struct's `WindowGroup` at `RootView()` instead of `ContentView()`.

One note: this simple embed is the same mechanism YouTube's own `iframe` player uses, so it's fine for personal/prototype use. If you later want to pull in view counts, titles, or search your channel programmatically, that requires the YouTube Data API and an API key — a bigger step, only worth it if you need that metadata.

Part B — SwiftData for session history

SwiftData is Apple's modern persistence framework — think of it as a lightweight ORM built into the OS, roughly the role Room plays for Android or an embedded ORM would in a Java app. No server, no schema migrations to write by hand for simple changes.

B1. Define the model

New file, `MeditationSession.swift`:

```

import Foundation
import SwiftData

```

```
@Model
class MeditationSession {
    var date: Date
    var durationMinutes: Int

    init(date: Date = .now, durationMinutes: Int) {
        self.date = date
        self.durationMinutes = durationMinutes
    }
}
```

`@Model` is doing a lot here — it makes this class persistable, observable, and queryable, similar to how a JPA `@Entity` annotation would in Java, but with SwiftUI hooked in directly.

B2. Register the container

In your `@main` App file (e.g. `MindfulTimerApp.swift`):

```
import SwiftUI
import SwiftData

@main
struct MindfulTimerApp: App {
    var body: some Scene {
        WindowGroup {
            RootView()
        }
        .modelContainer(for: MeditationSession.self)
    }
}
```

This sets up local storage and makes it available to every view below `RootView` via the environment — you don't pass it down manually, same idea as React Context.

B3. Save a session when the timer finishes

In `ContentView`, add two properties:

```
@Environment(\.modelContext) private var modelContext
@Query(sort: \MeditationSession.date, order: .reverse) private var
pastSessions: [MeditationSession]
```

`@Query` is a live-updating read — like a `useQuery` hook that automatically re-renders when the underlying data changes. No manual refetch needed.

Update your timer's tick logic to record a session exactly once, when it reaches zero:

```

.onReceive(timer) { _ in
    guard isRunning, secondsRemaining > 0 else { return }
    secondsRemaining -= 1
    if secondsRemaining == 0 {
        isRunning = false
        modelContext.insert(MeditationSession(durationMinutes:
selectedMinutes))
    }
}

```

B4. Show the history

Add this below your timer's `Button` in `ContentView`'s `body`:

```

if !pastSessions.isEmpty {
    List {
        Section("History") {
            ForEach(pastSessions) { session in
                HStack {
                    Text("\(session.durationMinutes) min")
                    Spacer()
                    Text(session.date.formatted(date: .abbreviated, time:
.shortened))
                    .foregroundColor(.secondary)
                }
            }
        }
    }
    .frame(maxHeight: 240)
}

```

Run the app, complete a short session (pick 3 min to test quickly), and you should see it appear in the list — and still be there after you quit and relaunch the app, since `SwiftData` persists to disk automatically.

What you've now got

- A timer that records completed sessions
- A track list with real thumbnails that opens a YouTube player
- Local persistence with zero manual file/database handling

Natural next steps from here

- Link a track to a session (`MeditationSession` could store `trackID: String?` so history shows *what* you meditated to, not just how long)
- Swap the tab bar icons/colors to match your Degrees of Freedom branding
- A simple stats view — total minutes this week — is just a `reduce` over `pastSessions`, same as you'd do in JS